

Breadth First Search

AIM: Implement the Breadth First Search algorithm to solve a given problem

CODE:

```
graph = {'A' : ['B', 'C'],
        'B' : ['D', 'E'],
        'C' : ['F'],
        'D' : [],
        'E' : ['F'],
        'F' : []
        }
visited = []
queue = []

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print(s)

        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

bfs(visited, graph, 'A')
```

OUTPUT:

	A
	B
	C
	D
	E
	F
>>>	

Iterative Depth First Search

AIM: Implement the Iterative Depth First Search algorithm to solve the same problem

CODE:

```
graph = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F'],
'D': [],
'E': ['G', 'H'],
'F': ['P'],
'G': ['T'],
'H': [],
'P': [],
'T': []
}
visited=[]
goal='F'
depth=0
def ids(graph, node, visited,goal,depth):
    if node not in visited:
        visited.append(node)

        for n in graph[node] :
            depth+=1
            print(depth)

            if goal in visited:
                print('goal reached')

                break
            else:
                print('n is ', n)
                print('visited is ',visited)

                ids(graph,n, visited,goal,depth)

    return visited
visited = ids(graph,'A', visited,goal,depth)
print(visited)
```

OUTPUT:

```
1
n is B
visited is ['A']
2
n is D
visited is ['A', 'B']
3
n is E
visited is ['A', 'B', 'D']
4
n is G
visited is ['A', 'B', 'D', 'E']
5
n is T
visited is ['A', 'B', 'D', 'E', 'G']
5
n is H
visited is ['A', 'B', 'D', 'E', 'G', 'T']
2
n is C
visited is ['A', 'B', 'D', 'E', 'G', 'T', 'H']
3
n is F
visited is ['A', 'B', 'D', 'E', 'G', 'T', 'H', 'C']
4
goal reached
['A', 'B', 'D', 'E', 'G', 'T', 'H', 'C', 'F']
>>> |
```

A* Search

AIM: Implement the A* Search algorithm for solving a pathfinding problem

CODE:

RMP CODE:

```
dict_hn={'Arad':336,'Bucharest':0,'Craiova':160,'Drobeta':242,'Eforie':161,
        'Fagaras':176,'Giurgiu':77,'Hirsova':151,'Iasi':226,'Lugoj':244,
        'Mehadia':241,'Neamt':234,'Oradea':380,'Pitesti':100,'Rimnicu':193,
        'Sibiu':253,'Timisoara':329,'Urziceni':80,'Vaslui':199,'Zerind':374}

dict_gn=dict(
Arad=dict(Zerind=75,Timisoara=118,Sibiu=140),
Bucharest=dict(Urziceni=85,Giurgiu=90,Pitesti=101,Fagaras=211),
Craiova=dict(Drobeta=120,Pitesti=138,Rimnicu=146),
Drobeta=dict(Mehadia=75,Craiova=120),
Eforie=dict(Hirsova=86),
Fagaras=dict(Sibiu=99,Bucharest=211),
Giurgiu=dict(Bucharest=90),
Hirsova=dict(Eforie=86,Urziceni=98),
Iasi=dict(Neamt=87,Vaslui=92),
Lugoj=dict(Mehadia=70,Timisoara=111),
Mehadia=dict(Lugoj=70,Drobeta=75),
Neamt=dict(Iasi=87),
Oradea=dict(Zerind=71,Sibiu=151),
Pitesti=dict(Rimnicu=97,Bucharest=101,Craiova=138),
Rimnicu=dict(Sibiu=80,Pitesti=97,Craiova=146),
Sibiu=dict(Rimnicu=80,Fagaras=99,Arad=140,Oradea=151),
Timisoara=dict(Lugoj=111,Arad=118),
Urziceni=dict(Bucharest=85,Hirsova=98,Vaslui=142),
Vaslui=dict(Iasi=92,Urziceni=142),
Zerind=dict(Oradea=71,Arad=75)
)
```

A*SEARCH CODE:

```
import queue as Q

from RMP import dict_gn
from RMP import dict_hn

start='Arad'
goal='Bucharest'
result = ' '

def get_fn(citystr):
    cities=citystr.split(',')
    hn=gn=0
    for ctr in range(0,len(cities)-1):
        gn=gn+dict_gn[cities[ctr]][cities[ctr+1]]
    hn=dict_hn[cities[len(cities)-1]]
    return (hn+gn)

def expand(cityq):
    global result
    tot,citystr,thiscity=cityq.get()
    if thiscity==goal:
        result=citystr+'::'+str(tot)
        return
    for cty in dict_gn[thiscity]:
        cityq.put((get_fn(citystr+","+cty),citystr+","+cty,cty))
    expand(cityq)

def main():
    cityq=Q.PriorityQueue()
    thiscity=start
    cityq.put((get_fn(start),start,thiscity))
    expand(cityq)
    print('The A* path with the total is: ')
    print(result)

main()
|
```

OUTPUT:

```
>>> | The A* path with the total is:
    | Arad,Sibiu,Rimnicu,Pitesti,Bucharest::418
```

Recursive Best-First Search

AIM: Implement the Recursive Best-First Search algorithm for the same problem

CODE:

```
import math

heuristics = {
    'Arad': 366, 'Zerind': 374, 'Oradea': 380, 'Sibiu': 253, 'Timisoara': 329,
    'Lugoj': 244, 'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160, 'Rimnicu Vilcea': 193,
    'Fagaras': 178, 'Pitesti': 98, 'Bucharest': 0, 'Giurgiu': 77, 'Urziceni': 80,
    'Hirsova': 151, 'Eforie': 161, 'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}

romania_map = {
    'Arad': {'Zerind': 75, 'Timisoara': 118, 'Sibiu': 140},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

class Node:
    def __init__(self, city, parent=None, g=0):
        self.city = city
        self.parent = parent
        self.g = g
        self.h = heuristics[city]
```

```

        self.parent = parent
        self.g = g
        self.h = heuristics[city]
        self.f = max(self.g + self.h, getattr(parent, 'f', 0))

def rbfs(node, goal, f_limit):
    print(f"Visiting {node.city}, f={node.f}, limit={f_limit}")

    if node.city == goal:
        return node, 0

    successors = []
    for neighbor, cost in romania_map[node.city].items():
        if node.parent and neighbor == node.parent.city:
            continue

        g = node.g + cost
        child = Node(neighbor, parent=node, g=g)
        successors.append(child)

    if not successors:
        return None, math.inf

    while True:
        successors.sort(key=lambda x: x.f)
        best = successors[0]

        if best.f > f_limit:
            return None, best.f

        alternative = successors[1].f if len(successors) > 1 else math.inf

        result, best.f = rbfs(best, goal, min(f_limit, alternative))

    if result is not None:
        return result, best.f

```

```

def extract_path(node):
    path = []
    while node:
        path.append(node.city)
        node = node.parent
    return path[::-1]

if __name__ == "__main__":
    start = 'Arad'
    goal = 'Bucharest'

    start_node = Node(start)
    result, _ = rbfs(start_node, goal, math.inf)

    if result:
        path = extract_path(result)
        print("\nPath found:")
        print(" -> ".join(path))
        print("Total cost:", result.g)
    else:
        print("No path found.")

```

OUTPUT:

```

Visiting Arad, f=366, limit=inf
Visiting Sibiu, f=393, limit=447
Visiting Rimnicu Vilcea, f=413, limit=417
Visiting Pitesti, f=415, limit=417
Visiting Fagaras, f=417, limit=418
Visiting Rimnicu Vilcea, f=418, limit=447
Visiting Pitesti, f=418, limit=447
Visiting Bucharest, f=418, limit=447

Path found:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total cost: 418
>>>

```

Decision Tree Learning

AIM:

- Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
- Evaluate the accuracy and effectiveness of the decision tree on test data.
- Visualize and interpret the generated decision tree.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        . . .
```

```
        y_pred,  
        target_names=iris.target_names  
    )  
|  
plt.figure(figsize=(12, 7))  
  
plot_tree(  
    model,  
    feature_names=iris.feature_names,  
    class_names=iris.target_names,  
    filled=True,  
    rounded=True  
)  
  
plt.title("Decision Tree - Iris Dataset")  
plt.show()
```

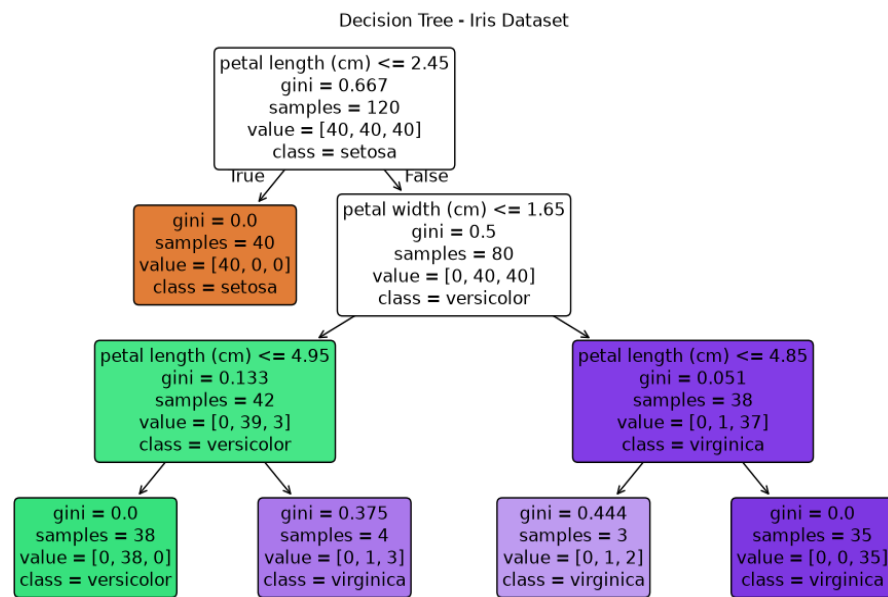
OUTPUT:

```
*IDLE Shell 3.12.10*
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\ranep\OneDrive\Documents\ML_practicals\Decision_Tree_Iris\decisiontre
Accuracy: 0.9666666666666667

Classification Report:
              precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10
  versicolor  1.00      0.90      0.95        10
   virginica   0.91      1.00      0.95        10

 accuracy      0.97
  macro avg    0.97      0.97      0.97        30
 weighted avg  0.97      0.97      0.97        30
```



Feed Forward Backpropagation Neural Network

AIM:

- Implement the Feed Forward Backpropagation algorithm to train a neural network.
- Use a given dataset to train the neural network for a specific task.
- Evaluate the performance of the trained network on test data.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    learning_rate_init=0.01,
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
y_pred = model.predict(X_test)

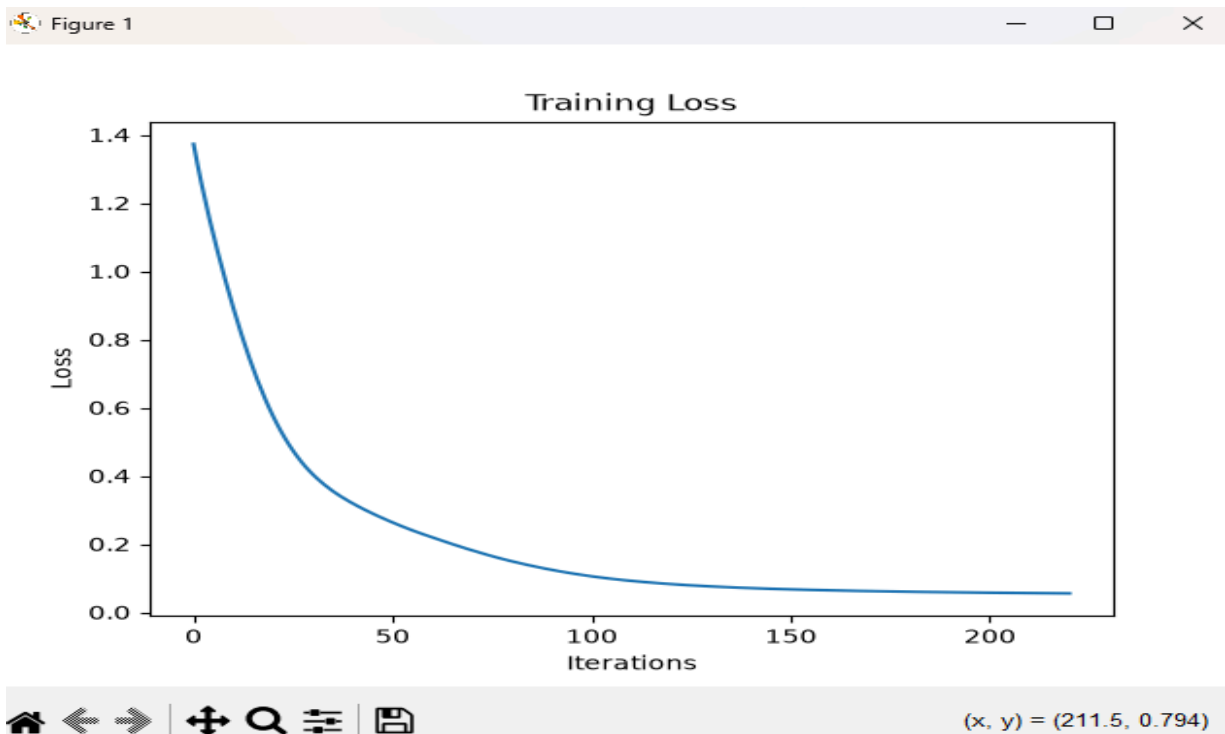
accuracy = accuracy_score(y_test,y_pred)
print("Acuuracy:",accuracy)

cm = confusion_matrix(y_test,y_pred)
print("Confusion Matrix:")
print(cm)

plt.plot(model.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Training Loss")
plt.show()
```

OUTPUT:

```
Acuuracy: 1.0
Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
>>> |
```



Support Vector Machines (SVM)

AIM:

- Implement the SVM algorithm for binary classification.
- Train an SVM model using a given dataset and optimize its parameters.
- Evaluate the performance of the SVM model on test data and analyze the results.

CODE:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

iris = load_iris()

X = iris.data
y = iris.target

print("Feature names:")
print(iris.feature_names)

print("\nTarget names:")
print(iris.target_names)

print("\nFirst five records:")
print(X[:5])

print("\nFirst five target values:")
print(y[:5])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = SVC(
    kernel='linear',
    C=1.0
)

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
```

```

    )

plt.figure(figsize=(8, 6))

for class_value in range(3):

    plt.scatter(
        X[y == class_value, 2],
        X[y == class_value, 3],
        label=iris.target_names[class_value]
    )

plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Iris Dataset - SVM")
plt.legend()
plt.show()

```

OUTPUT:

```

===== RESTART: C:\Users\ranep\OneDrive\Documents\ML_practicals\Decision
Feature names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target names:
['setosa' 'versicolor' 'virginica']

First five records:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]

First five target values:
[0 0 0 0 0]

Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]

Classification Report:

```

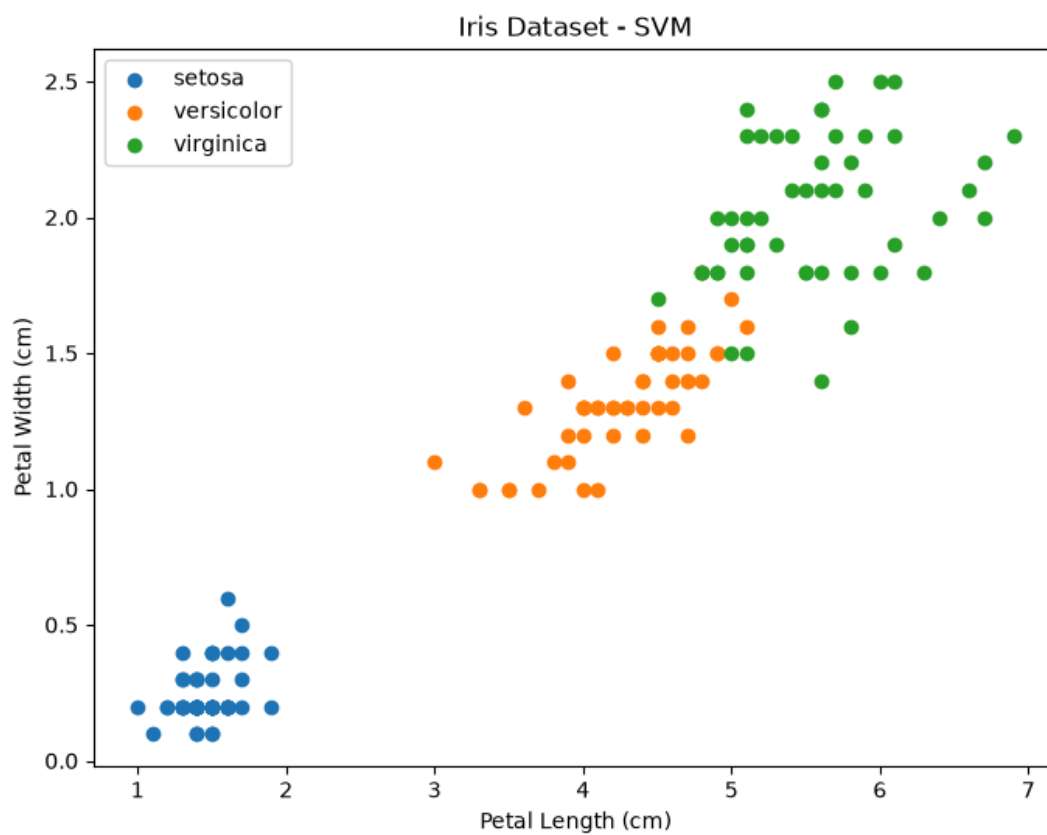
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```

>>> |

```

Figure 1



Adaboost Ensemble Learning

AIM:

- Implement the Adaboost algorithm to create an ensemble of weak classifiers.
- Train the ensemble model on a given dataset and evaluate its performance.
- Compare the results with individual weak classifiers.

CODE:

OUTPUT:

Naive Bayes' Classifier

AIM:

- Implement the Naive Bayes' algorithm for classification.
- Train a Naive Bayes' model using a given dataset and calculate class probabilities.
- Evaluate the accuracy of the model on test data and analyze the results.

CODE:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

iris = load_iris()

X = iris.data
y = iris.target

print("Feature names:")
print(iris.feature_names)

print("\nTarget names:")
print(iris.target_names)

print("\nFirst five records:")
print(X[:5])

print("\nFive five target values:")
print(y[:5])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = SVC(
    kernel='linear',
    C=1.0
)

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

new_flower=[[5.1,3.5,1.4,0.2]]
new_flower_scaled = scaler.transform(new_flower)
Prediction = model.predict(new_flower_scaled)
print("Predicted class", iris.target_names[Prediction[0]])

plt.figure(figsize=(8,6))

for class_value in range(3):

    plt.scatter(
        X[y == class_value, 2],
        X[y == class_value, 3],
        label=iris.target_names[class_value]
    )

plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("iris Dataset - SVM")
plt.legend()
plt.show()

```

OUTPUT:

```
Feature names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target names:
['setosa' 'versicolor' 'virginica']

First five records:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]

Five five target values:
[0 0 0 0 0]

Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]

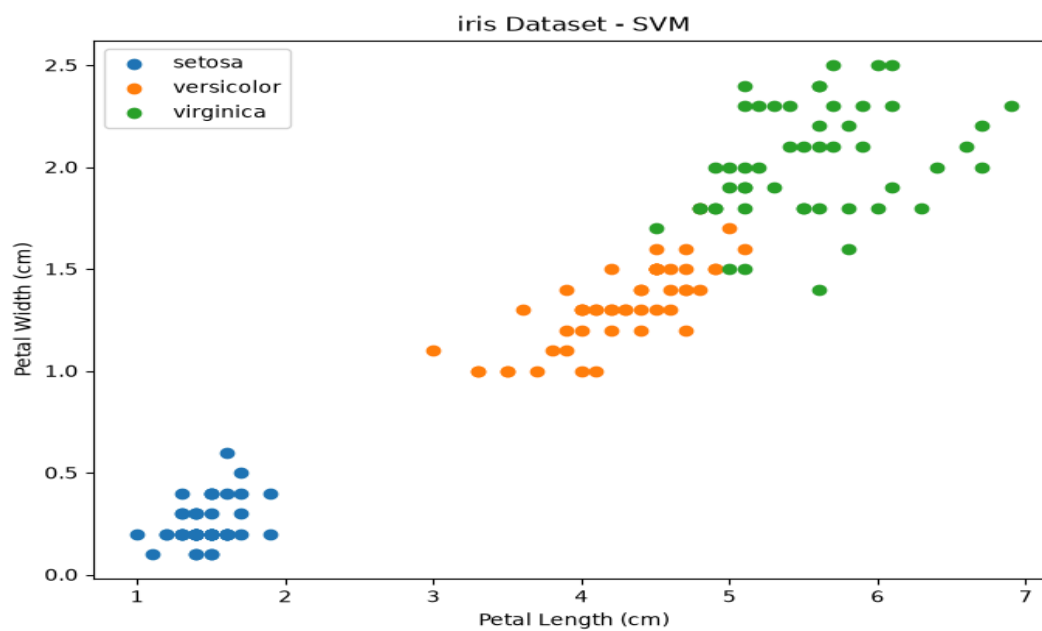
Classification Report:
              precision    recall  f1-score   support

   setosa               1.00      1.00      1.00        10
  versicolor            1.00      1.00      1.00        10
   virginica            1.00      1.00      1.00        10

 accuracy               1.00      1.00      1.00        30
 macro avg              1.00      1.00      1.00        30
 weighted avg           1.00      1.00      1.00        30

Predicted class setosa
```

Figure 1



K-Nearest Neighbors (K-NN)

AIM:

- Implement the K-NN algorithm for classification or regression.
- Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
- Evaluate the accuracy or error of the predictions and analyze the results.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris(as_frame=True)
df = iris.frame

X = df.drop(columns=['target'])
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))
```

```

X_vis = df[['sepal length (cm)', 'sepal width (cm)']].values
y_vis = df['target']

scaler_vis = StandardScaler()
X_vis_scaled = scaler_vis.fit_transform(X_vis)

knn_vis = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn_vis.fit(X_vis_scaled, y_vis)

import numpy as np

x_min, x_max = X_vis_scaled[:, 0].min() - 1, X_vis_scaled[:, 0].max() + 1
y_min, y_max = X_vis_scaled[:, 1].min() - 1, X_vis_scaled[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)

Z = knn_vis.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.coolwarm)

sns.scatterplot(
    x=X_vis_scaled[:, 0],
    y=X_vis_scaled[:, 1],
    hue=y_vis,
    palette="deep",
    s=60
)

plt.title("K-NN Decision Boundary (Iris Dataset, k=5)")
plt.xlabel("Sepal Length (scaled)")
plt.ylabel("Sepal Width (scaled)")
plt.show()

```

OUTPUT:

```
Accuracy: 1.0
```

```
Confusion Matrix:
```

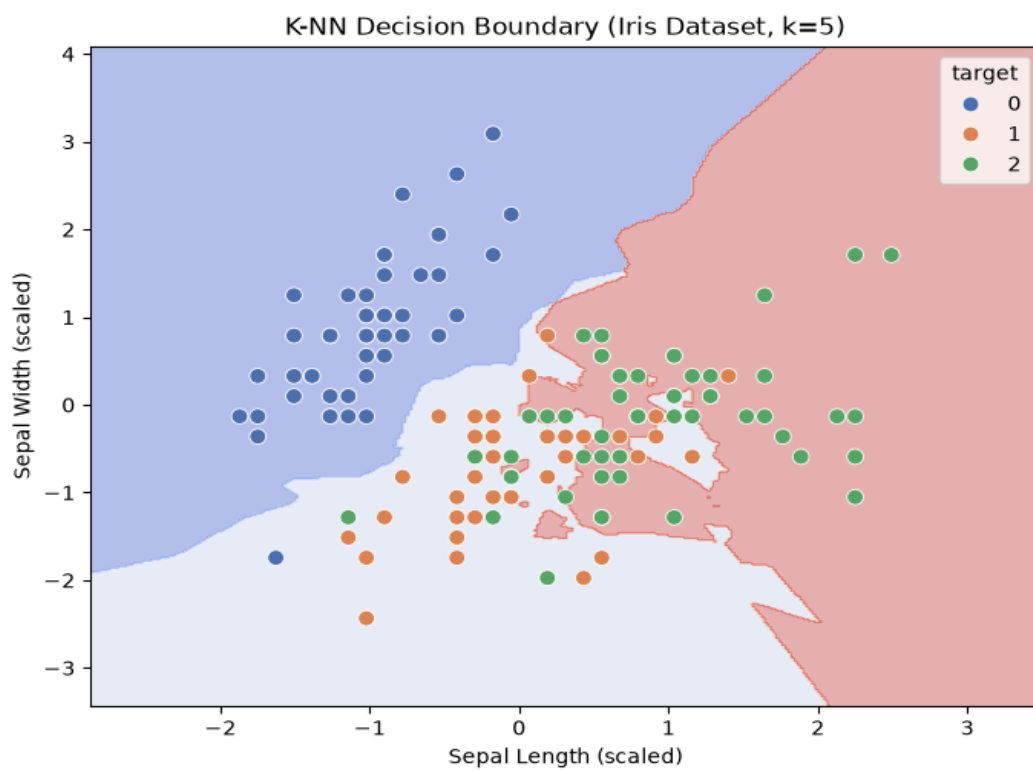
```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
>>> |
```

Figure 1



Association Rule Mining

AIM:

- Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.
- Generate association rules from the frequent itemsets and calculate their support and confidence.
- Interpret and analyze the discovered association rules.

CODE:

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
import pandas as pd

transactions = [
    ['Bread', 'Milk'],
    ['Bread', 'Butter', 'Juice', 'Eggs'],
    ['Milk', 'Butter', 'Juice', 'Coke'],
    ['Bread', 'Milk', 'Butter', 'Juice'],
    ['Bread', 'Milk', 'Butter', 'Coke']
]

te = TransactionEncoder()
te_data = te.fit(transactions).transform(transactions)

df = pd.DataFrame(te_data, columns=te.columns_)
print("Transaction Data:")
print(df)

frequent_itemsets = apriori(
    df,
    min_support=0.6,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets)

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.7
)
```

```
print("\nAssociation Rules:")
print(rules[['antecedents',
             'consequents',
             'support',
             'confidence']])
```

OUTPUT:

```
Transaction Data:
  Bread  Butter  Coke  Eggs  Juice  Milk
0   True   False  False  False  False  True
1   True    True  False   True   True  False
2  False    True   True  False   True   True
3   True    True  False  False   True   True
4   True    True   True  False  False   True

Frequent Itemsets:
  support  itemsets
0     0.8  frozenset({Bread})
1     0.8  frozenset({Butter})
2     0.6  frozenset({Juice})
3     0.8  frozenset({Milk})
4     0.6  frozenset({Butter, Bread})
5     0.6  frozenset({Milk, Bread})
6     0.6  frozenset({Butter, Juice})
7     0.6  frozenset({Butter, Milk})

Association Rules:
  antecedents  consequents  support  confidence
0  frozenset({Butter})  frozenset({Bread})    0.6      0.75
1  frozenset({Bread})  frozenset({Butter})    0.6      0.75
2  frozenset({Milk})  frozenset({Bread})    0.6      0.75
3  frozenset({Bread})  frozenset({Milk})    0.6      0.75
4  frozenset({Butter})  frozenset({Juice})    0.6      0.75
5  frozenset({Juice})  frozenset({Butter})    0.6      1.00
6  frozenset({Butter})  frozenset({Milk})    0.6      0.75
7  frozenset({Milk})  frozenset({Butter})    0.6      0.75
>>>|
```

